Laporan Tugas Besar 1

IF3070 Dasar Inteligensi Artifisial



Oleh

Kelompok 39 - yaa

Dastya	11523033
Stella Cometta Febriana	1822435
Nathan Pasha Athalla	18224053

Tautan *repository* GitHub:

https://github.com/daaysta/IF3070_G39_Yaaa

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO & INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi **2**

1. Deskripsi Persoalan **4**

1.1. Ringkasan Masalah 4

1.2. Variabel dan Representasinya 4

1.3. Objective Function 5

1.4. Constraint 6

1.5. State dan Move 6

2. Pembahasan **7**

2.1. Implementasi Algoritma Local Search 7

2.1.1. Komponen Umum 7

2.1.2. Hill-Climbing 9

2.1.2.1. Steepest Ascent Hill-Climbing 11

2.1.2.2. Stochastic Hill-Climbing 11

2.1.2.3. Hill-Climbing with Sideways Move 11

2.1.2.4. Random Restart Hill-Climbing 11

2.1.3. Simulated Annealing 11

2.1.4. Genetic Algorithm 11

2.2. Hasil Eksperimen 14

2.2.1. Steepest Ascent Hill-Climbing 15

2.2.1.1. Single Ship 15

2.2.2. Stochastic Hill-Climbing 15

2.2.2.1. Single Ship 15

2.2.3. Hill-Climbing with Sideways Move 16

2.2.3.1. Single Ship 16

2.2.4. Random Restart Hill-Climbing 16

2.2.4.1. Single Ship 16

2.2.5. Simulated Annealing 17

2.2.5.1. Single Ship..... 17
2.2.6. Genetic Algorithm..... 17
2.2.6.1. Single Ship..... 18
2.3. Analisis..... 25
3. Kesimpulan dan Saran..... 26
3.1. Kesimpulan..... 26
3.2. Saran..... 26
4. Pembagian Tugas..... 26
5. Referensi..... 26
6. Lampiran..... 27
6.1. Form Penggunaan Generative AI.....2

BAB I

DESKRIPSI PERSOALAN

1.1. Ringkasan Masalah

PT Owen Intim Line mengoperasikan kapal kargo pengangkut kendaraan. Pada tiap jadwal keberangkatan perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman, dan ETA yang berbeda. Penempatan manual membuat ruang geladak kurang termanfaatkan dan keuntungan tidak optimal.

Sistem pendukung keputusan yang dibangun harus memilih kendaraan yang diangkut serta posisi dan orientasinya di geladak, sehingga total ShippingFee maksimum dengan semua batasan terpenuhi. Karena ruang pencarian kombinatorial (subset kendaraan, koordinat, orientasi), persoalan diselesaikan dengan local search: Hill-Climbing (varian sideways move), Simulated Annealing, dan Genetic Algorithm.

Sistem yang dibangun memutuskan tiga hal: kendaraan mana yang diangkut, di koordinat mana kendaraan itu diletakkan pada geladak, dan dengan orientasi apa (*Horizontal* atau *Vertical*). Tujuannya memaksimalkan total *shipping fee* dari kendaraan yang terangkut, tanpa ada kendaraan yang saling bertumpuk, keluar batas geladak, atau melebihi kapasitas berat kapal.

Instance dibangkitkan oleh make_instance di src/runner.py dengan seed 42 dan disimpan di output/instance.json. Isinya satu kapal berukuran 8 x 12 sel berkapasitas 100 unit, dan 30 kendaraan. Jika semua kendaraan digabung, fee-nya 1594, beratnya 189, dan luasnya 176 sel, sedangkan geladak hanya 96 sel. Berat dan luas sama-sama melebihi batas, jadi tidak semua kendaraan bisa diangkut. Implementasi saat ini hanya mendukung satu kapal.

1.2. Variabel dan Representasinya

Entitas	**Atribut**	**Representasi di Program**
Vehicle	id	Kunci *dict* vehicles, *string* (misal “V01”), juga kunci placement. Id tidak boleh duplikat.
	w	vehicles[id]["w"], *int* (satuan *grid*)
	l	vehicles[id]["l"], *int*
	fee	vehicles[id]["fee"], *int* (juta rupiah)
	weight	vehicles[id]["weight"], *int* (*unit*)
	eta	vehicles[id]["eta"], *int* (hari). Dimuat, tetapi belum dipakai oleh *objective function*.
	Orientasi & posisi	placement[id] = {"ship", "x", "y", "orient"}. orient bernilai "H" (ukuran w x l) atau "V" (ukuran ditukar jadi l x w). x, y adalah sel kiri bawah, bilangan bulat mulai dari 1.
	di luar kapal	placement[id]["ship"] = OUTSIDE (konstanta -1) dan x = y = None. Orientasi tetap disimpan.
Ships	id	Kunci *dict* ships, *int*. Tidak boleh sama dengan OUTSIDE atau duplikat.
	w, l	ships[id]["w"], ships[id]["l"]
	capacity	ships[id]["capacity"]. Beban kapal tidak disimpan, dihitung dari placement oleh ship_load(ship).

Satu *state* adalah pemetaan placements = {id: (x, y, orient)} untuk kendaraan yang diangkut dan None untuk kendaraan di luar kapal. *Grid* geladak, total berat, dan total *fee* disimpan sebagai *cache* di objek State. Instance eksperimen dibangkitkan dengan *seed* 42: geladak 8 x 12 (96 sel), kapasitas berat 100, 30 kendaraan (w 1 sampai 3, l 2 sampai 4, *fee* 10 sampai 100, berat 2 sampai 10, ETA 1 sampai 10).

Total seluruh kendaraan: *fee* 1594, berat 189, luas 176 sel. Berat (189 > 100) dan luas (176 > 96) sama-sama membatasi, sehingga tidak semua kendaraan dapat diangkut. *State* direpresentasikan oleh kelas DockState (src/model/state.py). Untuk visualisasi, build_grids() mengubah state menjadi *grid* numpy per kapal (sel kosong bernilai -1, sel terisi berisi indeks kendaraan).

1.3. *Objective Function*

Fungsi ini dimaksimalkan. *State.value()* mengembalikan *total_fee* yang disimpan dalam *cache.* Berbeda dari *template, value()* tidak memberi penalti otomatis pada state tidak valid, karena semua operator hanya menghasilkan state yang feasible. *Objective* yang dimaksimalkan semua algoritma adalah total *shipping fee* kendaraan yang berada di kapal, dan bernilai 0 bila ada constraint yang dilanggar:

$$f(s) = \begin{cases} \sum_{v \in P(s)} \text{fee}_v & \text{jika } s \text{ feasible} \\ 0 & \text{jika } s \text{ melanggar constraint} \end{cases}$$

Dengan P(s) adalah himpunan kendaraan yang ditempatkan di salah satu kapal pada *state* s (ship != OUTSIDE). Fungsinya adalah evaluate di src/model/objective.py, dengan nilai 0 untuk *state* tidak *feasible*. Akibatnya semua *state* yang melanggar sama nilainya, sehingga algoritma tidak punya gradien untuk keluar dari wilayah tidak *feasible*. Itu sebabnya inisialisasi dan GA dibuat selalu menghasilkan state *feasible*

1.4 Constraint

Tiga *constraint* dari spesifikasi dimodelkan sebagai tiga fungsi pemeriksa di src/model/constraints.py, lalu digabung oleh is_feasible:

- **Seluruh sel kendaraan berada di dalam batas geladak;** kendaraan di luar kapal berada seluruhnya di luar. (has_out_of_bounds): sel kiri bawah minimal (1, 1) dan sel kanan atas x + w - 1 <= W, y + l - 1 <= L, dengan w dan l sesuai orientasi (fungsi footprint).
- **Tidak ada *overlap*:** satu sel geladak ditempati paling banyak satu kendaraan. tiap pasangan kendaraan di kapal yang sama dicek sekali

dengan uji irisan dua persegi panjang (rectangles_overlap) di sumbu x dan y sekaligus. Dua kendaraan yang hanya bersentuhan sisi tidak dihitung *overlap*.

- **Kapasitas** (is_overweight): jumlah berat kendaraan di tiap kapal tidak boleh melebihi capacity kapal itu.

Constraint ditegakkan secara **konstruktif** (*feasible by construction*): **State.can_place** memeriksa batas, ***overlap***, dan kapasitas sebelum kendaraan ditempatkan, sehingga *initial state* dan semua *neighbor* selalu valid dan tidak perlu fungsi penalti. State.is_valid() dipakai sebagai verifikasi independen di tes dan saat memulihkan hasil dari JSON.

Pelanggaran ditangani dengan tidak membuang *state*: *neighbor* yang melanggar tetap dibangkitkan dan bernilai 0 di evaluate. Untuk membangun state *feasible* tersedia can_place, yang mengecek satu kendaraan di satu posisi terhadap ketiga *constraint* dengan menganggap posisi lama kendaraan itu kosong. can_place dipakai oleh *holl-climbing* (neighbor_value)

1.4. *State* dan *Move*

***Initial state*:** *State.random_state() m*engacak urutan kendaraan, lalu mencoba menempatkan tiap kendaraan di posisi dan orientasi acak (maksimal 30 percobaan). Kendaraan yang tidak mendapat posisi valid tetap di luar kapal.**Move (neighbor)**, sesuai spesifikasi:

1. Memindahkan satu kendaraan ke koordinat berbeda.
2. Mengubah orientasi kendaraan.
3. Menukar dua kendaraan (placement beserta orientasinya). Bila salah satu kendaraan di luar kapal, pertukaran mengganti kendaraan yang diangkut.

DockState.initial_state membuat *state* acak yang selalu *feasible*. Tiap kendaraan diberi orientasi acak dan kapal acak (atau OUTSIDE). Bila ukurannya tidak muat,

orientasi dibalik. Lalu koordinat acak dicoba sampai 50 kali dengan can_place. Kendaraan yang gagal tetap OUTSIDE.

***Final state*:** Pemetaan semua kendaraan (di dalam dan di luar kapal) beserta nilai *objective*-nya. SearchRecorder.finish menyimpannya ke JSON (*state* awal, *state* akhir, *state feasible* terbaik, riwayat *objective*, iterasi, durasi), dan build_grids menyiapkan *grid* geladak untuk digambar.

***Move*:** Tiga *move* ada di src/moves/moves.py:

Move	**Fungsi**	**Cara Kerja**
Tukar dua kendaraan	swap_move	Menukar (*ship*, x, y) dua kendaraan, orientasi tetap milik masing-masing. Bila salah satu OUTSIDE, yang di luar masuk dan yang di dalam keluar.
Pindah koordinat	place_move	Memindahkan satu kendaraan ke (*ship*, x, y) lain: dari luar ke dalam, antar kapal, atau di kapal yang sama. Posisi yang akan keluar geladak tidak pernah dibuat.
Ubah orientasi	rotate_move	Membalik H dan V, titik kiri bawah tetap. Kendaraan persegi (w = l) dilewati karena rotasinya tidak mengubah apa pun.

random_neighbor memilih jenis *move* dengan bobot swap 0.4, relocate 0.4, rotate 0.2. all_neighbors membangkitkan seluruh *neighbor* sebagai generator untuk algoritma yang butuh semua *neighbor*.

BAB II.

PEMBAHASAN

2.1 Implementasi Algoritma *Local Search*

2.1.1 Komponen Umum

Semua *state* yang dibangkitkan oleh *initial state* dan oleh operator GA diperlakukan sebagai objek baru (copy), jadi *state* asal tidak berubah ketika *neighbor* dibuat. Data statis kendaraan dan kapal dibagi antar *state* sehingga penyalinan murah.

File	**Fungsi/Kelas**	**Peran**
src/models.py	Vehicle, Ship(dataclass)	Mendefinisikan atribut kendaraan (id, w, l, fee, weight, eta) dan kapal (w, l, max_capacity).
src/state.py	get_dim, State, konstanta HORIZONTAL, VERTICAL, ORIENTS	get_dim menghitung ukuran kendaraan (kolom, baris) menurut orientasi. Statemenyimpan *placement* (id → (x, y, orient) atau None bila di luar kapal) beserta cache grid, total berat, dan total fee. Metode: copy, footprint, value, random_state (*state* awal acak yang *feasible*), from_plan, inside_ids, outside_ids, to_dict, render_text.
src/state.py (bagian *constraint*)	is_valid, can_place	Pengecekan *constraint* (lihat 1.4). is_valid memeriksa batas geladak, *no overlapping*, orientasi, dan kapasitas berat pada seluruh state. can_placememeriksa apakah satu kendaraan boleh ditaruh di posisi tertentu tanpa melanggar *constraint*.
src/state.py (bagian *objective*)	value, atribut total_fee	Menghitung *objective* (lihat 1.3), yaitu total fee kendaraan yang berada di dalam kapal. Nilai disimpan di cache dan diperbarui saat kendaraan

		ditaruh atau dikeluarkan.
src/state.py (bagian *move*)	random_neighbor, all_neighbors, _moved, _swapped, _remove, _place	Pembangkit *neighbor*. *_moved m*emindahkan atau mengubah orientasi kendaraan (termasuk mengeluarkannya dari kapal), _swapped menukar dua kendaraan. random_neighbor mengambil satu *neighbor* valid secara acak (maks. 50 percobaan). all_neighbors membangkitkan seluruh *neighbor*.
src/result.py	SearchResult	Pencatat hasil yang dipakai semua algoritma: *state* awal dan akhir*, history obj*ective per iterasi, jumlah iterasi, durasi, dan extras. to_dict menghasilkan *dict* siap disimpan ke JSON.
src/runner.py	make_instance, load_instance, run_hc, run_sa, run_ga, write_summary, main	Menjalankan eksperimen: HC 3×, SA 3×, GA 18× (dua kelompok konfigurasi). *Run* ke-k memakai seed base_seed + k, sehingga *state* awal dan hasilnya bisa diulang. Hasil disimpan di output/<algoritma>/ beserta summary.csv dan summary.json.
src/visualize.py	plot_objective, draw_deck, gambar_state, plot_sa, save_all	Visualisasi: plot *objective* terhadap iterasi, gambar geladak (*state* awal/akhir) dengan tabel kendaraan di luar kapal, dan plot khusus SA.
src/plot_results.py	restore, build	Membaca ulang JSON hasil, memvalidasi *state* hasil pemulihan (is_valid dan kecocokan nilai), lalu meregenerasi PNG, *trace* CSV SA, ringkasan, dan grafik perbandingan di output/plots/.
src/test_state.py, src/test_sa.py	SATest (di test_sa.py)	Tes otomatis untuk State dan SA.

Catatan desain komponen umum.

- copy() menyalin placement, grid, berat, dan fee, tetapi data kendaraan dan kapal dipakai bersama antar *state*(new.vehicles = self.vehicles). Karena itu pembuatan *neighbor* tidak mengubah *state* asal dan penyalinannya murah.
- Semua operator (random_state, _moved, _swapped, from_plan) hanya menghasilkan *state* yang *feasible* lewat can_place. Konsekuensinya value() tidak perlu memberi penalti pada *state* tidak valid.
- Pada _swapped, bila salah satu kendaraan berada di luar kapal, pertukaran mengganti kendaraan yang diangkut. Hal ini yang membuat pemilihan kendaraan ikut dioptimasi, bukan hanya penataannya.
- Representasi tersebut menjadi dasar untuk menunjukkan tiga hal yang diminta spesifikasi pada *final state*: keadaan kendaraan di dalam dan di luar (inside_ids, outside_ids), keadaan geladak (grid, render_text), dan nilainya (value).

2.1.2. Hill-Climbing

Implementasi ada di src/hill_climbing.py. **Hanya satu varian yang diimplementasikan, yaitu *Hill-Climbing with Sideways Move.***

File	Fungsi/Kelas	Peran
src/hill_climbing.py	hill_climbing_sideways	Algoritma HC with sideways move. Parameter: initial, max_sideways, max_iter, reset_on_improve, rng. Mengembalikan SearchResult.
src/state.py	all_neighbors	all_neighbors membangkitkan seluruh neighbor (pindah posisi, ubah orientasi, keluarkan kendaraan, dan tukar dua kendaraan). value mengembalikan total fee.

src/result.py	SearchResult	Menyimpan state awal dan akhir, history objective, jumlah iterasi, durasi, dan data tambahan
src/runner.py	run_hc	Menjalankan 3 *run* dengan HC_MAX_SIDEWAYS = 100, seed 42 + k.
src/visualize.py	Plot_objective, gambar_state	Membuat grafik *objective* terhadap iterasi dan darah awal dan akhir.

2.1.2.1. Steepest Ascent Hill-Climbing

Tidak diimplementasikan sebagai fungsi terpisah. Secara logika, max_sideways = 0 pada hill_climbing_sidewaysberperilaku seperti steepest ascent. Namun varian ini belum pernah dijalankan sebagai eksperimen, jadi di laporan cukup ditulis “tidak diimplementasikan, setara max_sideways = 0” tanpa hasil.

2.1.2.2. Stochastic Hill-Climbing

Tidak diimplementasikan.

2.1.2.3. Hill-Climbing with Sideways Move]

Pada tiap iterasi, algoritma mengevaluasi **seluruh** *neighbor* dari *state* sekarang lewat all_neighbors(), lalu memilih yang bernilai tertinggi. Bila ada beberapa *neighbor* terbaik dengan nilai sama, satu dipilih secara acak memakai *reservoir sampling* (rng.random() < 1/n_best), sehingga tiap kandidat berpeluang sama tanpa menyimpan semuanya dalam list.

- Nilai terbaik **lebih tinggi** dari sekarang: pindah dan hitungan sideways berurutan di-reset ke 0.
- Nilai terbaik **sama**: pindah (*sideways move*) selama batas belum tercapai.
- Nilai terbaik **lebih rendah**: berhenti (*local optimum*). HC tidak pernah menerima penurunan.

Parameter: max_sideways = 100, max_iter = 10.000, reset_on_improve = True. Dengan reset_on_improve = True, batas 100 berlaku pada rangkaian *sideways* **berturut-turut**, bukan total selama pencarian. Total *sideways* tetap dicatat di extras["sideways_used"]. **Kriteria berhenti** (extras["stop_reason"]):

stop_reason	Kondisi
max_sideways	Rangkaian sideways berturut-turut mencapai 100
local_optimum	*Neighbor* terbaik lebih buruk dari *state* sekarang
no_neighbor	Tidak ada *neighbor*
max_iter	Iterasi mencapai 10.000

state awal dan akhir, nilai *objective* akhir, history (indeks 0 = *state*awal, lalu satu nilai per perpindahan), banyak iterasi (termasuk langkah *sideways*), durasi (perf_counter), stop_reason, dan sideways_used.

Source code:

```
def hill_climbing_sideways(initial, max_sideways=100, max_iter=10_000,
        best, best_val, n_best = None, None, 0

        for nb in current.all_neighbors():
            v = nb.value()
            if best_val is None or v > best_val:
                best, best_val, n_best = nb, v, 1
            elif v == best_val:
                n_best += 1
                if rng.random() < 1.0 / n_best:
                    best = nb

        if best is None:
            stop_reason = "no_neighbor"
            break
        if best_val < cur_val:
            stop_reason = "local_optimum"
            break

        if best_val == cur_val:
            used = streak if reset_on_improve else sideways_used
            if used >= max_sideways:
                stop_reason = "max_sideways"
                break
            streak += 1
            sideways_used += 1
        else:
            streak = 0

        current = best
        iterations += 1
        history.append(current.value())

    return SearchResult(
        name="hc_sideways",
        params={"max_sideways": max_sideways, "max_iter": max_iter,
                "reset_on_improve": reset_on_improve},
        initial_state=initial,
        final_state=current,
        history=history,
        iterations=iterations,
        duration=time.perf_counter() - start,
        extras={"stop_reason": stop_reason, "sideways_used": sideways_used},
    )
```

```python
def hill_climbing_sideways(initial, max_sideways=100, max_iter=10_000,
        best, best_val, n_best = None, None, 0

        for nb in current.all_neighbors():
            v = nb.value()
            if best_val is None or v > best_val:
                best, best_val, n_best = nb, v, 1
            elif v == best_val:
                n_best += 1
                if rng.random() < 1.0 / n_best:
                    best = nb

        if best is None:
            stop_reason = "no_neighbor"
            break
        if best_val < cur_val:
            stop_reason = "local_optimum"
            break

        if best_val == cur_val:
            used = streak if reset_on_improve else sideways_used
            if used >= max_sideways:
                stop_reason = "max_sideways"
                break
            streak += 1
            sideways_used += 1
        else:
            streak = 0

        current = best
        iterations += 1
        history.append(current.value())

    return SearchResult(
        name="hc_sideways",
        params={"max_sideways": max_sideways, "max_iter": max_iter,
                "reset_on_improve": reset_on_improve},
        initial_state=initial,
        final_state=current,
        history=history,
        iterations=iterations,
        duration=time.perf_counter() - start,
        extras={"stop_reason": stop_reason, "sideways_used": sideways_used},
    )
```

Implementasi.

- Satu iterasi HC mahal karena memeriksa seluruh *neighbor*. Ruang *neighbor* ini besar: tiap kendaraan bisa dipindah ke semua posisi yang muat, diputar, atau dikeluarkan, ditambah semua pasangan kendaraan untuk *swap*. Sebagai gantinya nilai naik sangat cepat di awal (lihat grafik objective HC).

- Banyak *neighbor* bernilai sama (mis. menggeser kendaraan di ruang kosong tidak mengubah fee). Itulah *plateau*yang membuat *sideways move* relevan, dan alasan ketiga *run* berhenti karena max_sideways dan bukan local_optimum.
- Karena max_iter 10.000 tidak pernah tercapai (ketiga *run* berhenti di sekitar 113–124 iterasi), batas yang benar-benar mengatur pencarian adalah max_sideways.

2.1.2.4. Random Restart Hill-Climbing

Tidak diimplementasikan (tidak ada parameter maximum restart).

2.1.3. Simulated Annealing

Implementasi ada di src/simulated_annealing.py.

File	Fungsi/Kelas	Peran
src/simulated_annealing.py	simulated_annealing	Algoritma SA lengkap: validasi parameter, penjadwalan suhu, keputusan penerimaan, *best state*, dan deteksi stagnasi.
src/state.py	random_neighbor	Mengambil satu *neighbor* acak yang valid sebagai proposal.
src/runner.py	run_sa	Menjalankan 3 *run* dengan parameter bawaan, seed 42 + k.
src/visualize.py	plot_sa	Plot e^(ΔE/T) terhadap iterasi, jadwal temperatur, panjang stagnasi, dan distribusi keputusan penerimaan.
src/test_sa.py	SATest	Tes penerimaan langkah turun, penolakan pada suhu rendah, hitungan stagnasi, dan reproduksibilitas.

Fungsi penjadwalan suhu. Geometrik: T ← T × cooling_rate setelah tiap iterasi, dengan T0 = 100 dan cooling_rate = 0,995. Pada sumbu log, suhu terlihat sebagai garis lurus.

Peluang penerimaan. Pada setiap iterasi, satu neighbor acak dibangkitkan sebagai proposal, lalu dihitung ΔE = fee kandidat − fee current.

Jika $\Delta E \geq 0$ (lebih baik atau sama), proposal selalu diterima.

Jika $\Delta E < 0$, proposal diterima dengan peluang $e^{(\Delta E/T)}$, yaitu ketika bilangan acak dalam rentang [0, 1) lebih kecil dari peluang tersebut.

Nilai $e^{(\Delta E/T)}$ di setiap iterasi disimpan di trace untuk plot penerimaan terhadap iterasi.

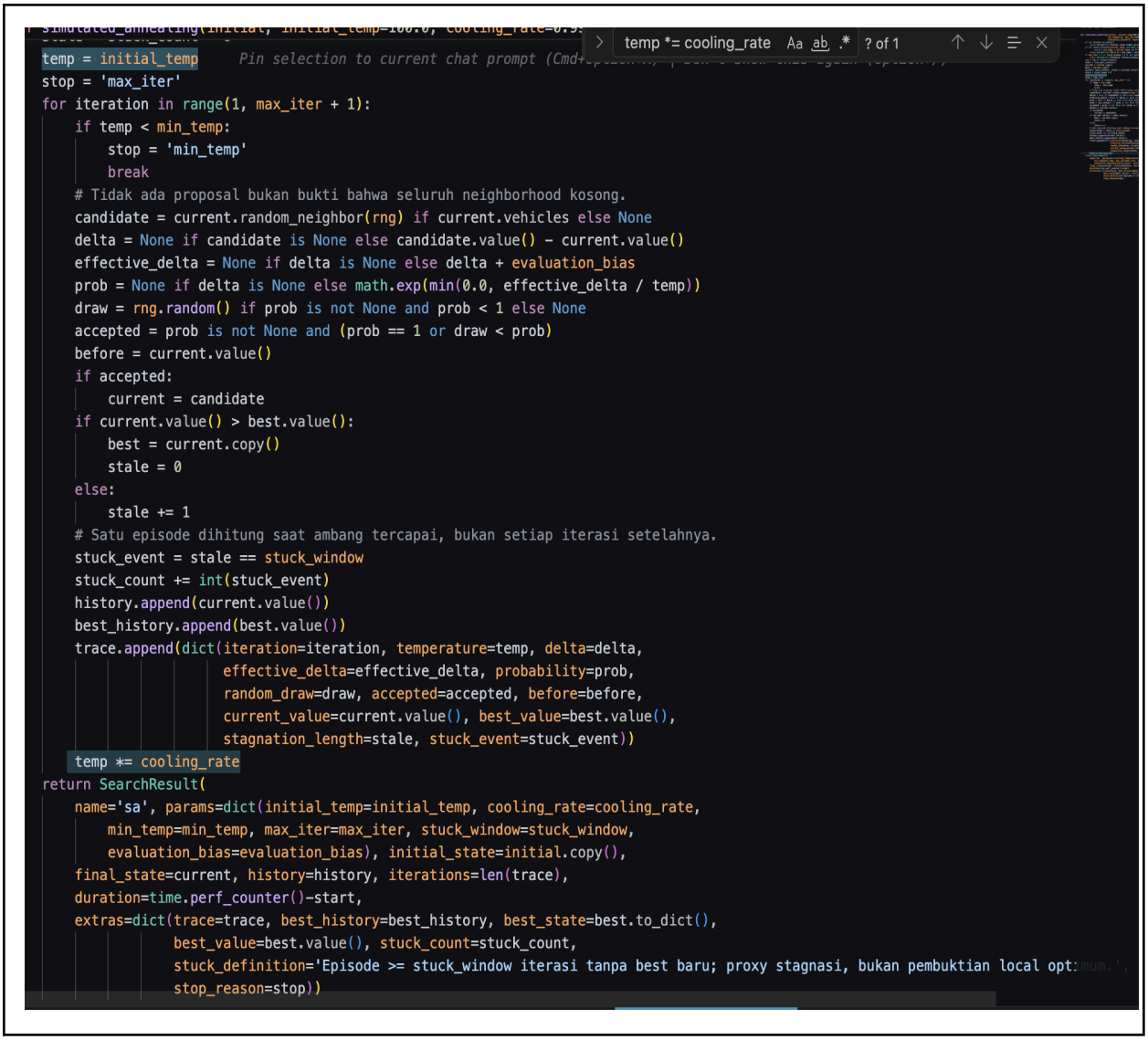
Parameter:

Parameter	**Nilai**
initial_temp	100
cooling_rate	0,995
min_temp	0,01
max_iter	5.000
stuck_window	100

Kriteria berhenti. T < min_temp (stop_reason = 'min_temp') atau iterasi mencapai max_iter. Dengan parameter di atas, suhu turun di bawah 0,01 setelah **1.838 iterasi**, sehingga ketiga *run* berhenti karena min_temp, bukan max_iter.

Frekuensi *stuck*. Variabel stale menghitung iterasi berturut-turut tanpa *best* baru. Satu episode *stuck* dihitung **sekali** saat stale == stuck_window (100), dan episode berikutnya baru dihitung setelah ada perbaikan *best*. Jumlahnya disimpan di extras["stuck_count"]. Ini **indikator stagnasi, bukan bukti *local optimum***, karena kode tidak memeriksa seluruh *neighborhood*. Definisi ini sudah ditulis di extras["stuck_definition"].

State akhir vs. best. final_state adalah *current* terakhir, sedangkan best_state disimpan terpisah di extras. Pada ketiga *run* nilainya kebetulan sama.



Analisis implementasi.

- Pada suhu tinggi, proposal yang menurunkan nilai sering diterima. Kurva *current* turun-naik, sedangkan *best* tidak pernah turun (Gambar 3).
- Banyak langkah diterima karena delta = 0 (1.285–1.350 per *run*). Seperti pada HC, banyak *neighbor* yang tidak mengubah fee.
- Penerimaan langkah turun hanya 33–48 kali per *run*, tetapi cukup membuktikan mekanisme eksplorasinya bekerja.
- SA sangat murah per iterasi karena hanya membangkitkan satu *neighbor*, bukan seluruhnya seperti HC.

2.1.4. Genetic Algorithm

Implementasi ada di src/ga.py.

File	Fungsi/Kelas	Peran
src/ga.py	pilih_parent, crossover, mutasi, genetic_algorithm	Seleksi *tournament*, *crossover* satu titik, mutasi, dan *loop* generasi.
src/state.py	random_state, from_plan, random_neighbor	Pembangkit populasi awal, pembangun anak yang selalu *feasible*, dan operator mutasi.
src/runner.py	_ga_runner, run_ga	Membungkus *dict* hasil GA menjadi SearchResult dan menjalankan 18 *run* (dua kelompok × tiga konfigurasi × tiga *run*).

Parameter dan konfigurasi. Dua parameter yang diubah: jumlah_populasi dan iterasi (satu iterasi = satu generasi). Parameter tetap: mutation_rate = 0,3, elite = 2, ukuran *tournament* k = 3.

Kelompo eksperimen	Kontrol	Variasi
ga_iter	Populasi 30	Generasi 50, 100, 200
ga_pop	Generasi 100	Populasi 10, 30, 50

Masing-masing dijalankan 3 kali, jadi 18 *run*. Konfigurasi pop30/gen100 muncul di kedua kelompok dengan seed sama, sehingga hasilnya identik dan bukan enam replikasi independen.

Komponen Genetic Algorithm:

- **Individu/kromosom:** satu State, dengan placement tiap kendaraan sebagai gen. Tidak ada konversi ke *list*terpisah.
- **Fitness:** State.value(), yaitu total fee. Tidak ada penalti karena semua individu *feasible*.
- **Populasi awal:** jumlah_populasi *state* dari State.random_state. *State* awal yang dilaporkan adalah individu terbaik populasi awal.
- **Seleksi:** *tournament* (pilih_parent). Ambil hingga 3 individu acak, yang bernilai tertinggi menang.
- **Crossover:** satu titik. Kendaraan [0, k) mengikuti parent A dan sisanya parent B. Anak dibangun lewat State.from_plan dengan **urutan pemasangan diacak**. Kendaraan yang bentrok atau melebihi kapasitas dibiarkan di luar kapal, sehingga anak selalu *feasible* tanpa fungsi *repair* terpisah.
- **Mutasi:** dengan peluang 0,3, anak diubah satu langkah random_neighbor (pindah, putar, atau tukar). Bila tidak ada proposal, individu dipakai apa adanya.
- **Elitism:** 2 individu terbaik dibawa langsung ke generasi berikutnya, sehingga nilai maksimum populasi tidak pernah turun.
- **Kriteria berhenti:** jumlah generasi tetap. state_akhir adalah individu terbaik sepanjang pencarian (terbaik_global).
- **Pencatatan:** tiap generasi (termasuk generasi 0) mencatat nilai maksimum (riwayat_max) dan rata-rata populasi (riwayat_rata), sesuai spesifikasi poin 4c.

Source code

```python
import random
import time

from state import State


def pilih_parent(populasi, rng, k=3):
    # tournament: ambil k acak, yang value-nya paling besar menang
    calon = rng.sample(populasi, min(k, len(populasi)))
    return max(calon, key=lambda s: s.value())


def crossover(a, b, rng):
    # one-point: sebagian kendaraan ikut parent a, sisanya parent b
    ids = list(a.vehicles)
    if len(ids) < 2:
        return a
    titik = rng.randint(1, len(ids) - 1)
    plan = {}
    for i, vid in enumerate(ids):
        induk = a if i < titik else b
        plan[vid] = induk.placements[vid]

    urutan = ids[:]
    rng.shuffle(urutan)
    return State.from_plan(a.vehicles, a.ship, plan, order=urutan)


def mutasi(s, rng):
    baru = s.random_neighbor(rng)
    return baru if baru else s


def genetic_algorithm(vehicles, ship, jumlah_populasi, iterasi,
                      mutation_rate=0.3, elite=2, seed=None):
    mulai = time.time()
    rng = random.Random(seed)

    populasi = [State.random_state(vehicles, ship, rng)
                for _ in range(jumlah_populasi)]
    state_awal = max(populasi, key=lambda s: s.value())
    terbaik_global = state_awal
```

Analisis implementasi.

- Karena tanpa *repair* eksplisit, anak hasil *crossover* cenderung kehilangan sebagian kendaraan (yang bentrok dibuang). Ini membuat *crossover* sedikit "destruktif", dan mutasi dengan random_neighbor yang membantu menambah kendaraan kembali.
- *Tournament selection* dipilih karena tidak bermasalah bila fitness sama atau nol. Berbeda dengan *roulette wheel*yang membutuhkan penanganan khusus saat total fitness 0.

- *State* awal GA adalah maksimum populasi, bukan satu *state* acak. Jadi kenaikan *objective* GA tidak setara langsung dengan HC/SA yang mulai dari satu *state* acak. Hal ini perlu disebut saat membandingkan.
- GA tidak memakai SearchResult secara langsung. runner.py mengonversi *dict* hasilnya (riwayat_max → history, riwayat_rata → extras["avg_history"]).

2.2 Hasil Eksperimen

Setup umum. Instance: 30 kendaraan, geladak 8 × 12 (96 sel), kapasitas berat 100, total fee seluruh kendaraan 1.594 juta rupiah (output/instance.json). Semua eksperimen dijalankan dari root project:

Run ke-k memakai seed 42 + k (43, 44, 45), sehingga *state* awal dan hasil bisa diulang. State awal HC dan SA pada *run* yang sama identik (798, 764, 982). Durasi HC dan SA diukur dengan time.perf_counter(), sedangkan GA dengan time.time(). Seluruh 24 hasil dinyatakan valid (valid = True di summary.csv). Satuan objective adalah juta rupiah.

2.2.1. Steepest Ascent Hill-Climbing

Tidak diimplementasikan sebagai fungsi terpisah, sehingga tidak ada hasil eksperimen. Secara perilaku, hill_climbing_sideways dengan max_sideways = 0 setara dengan steepest ascent: pencarian berhenti begitu neighbor terbaik tidak lebih baik dari state sekarang. Konfigurasi ini tidak dijalankan sebagai eksperimen.

2.2.2 Single Ship

Tidak diimplementasikan sebagai fungsi terpisah, sehingga tidak ada hasil eksperimen. Secara logika, hill_climbing_sideways dengan max_sideways = 0 setara dengan varian ini, tetapi konfigurasi tersebut belum pernah dijalankan

2.2.2 Stochastic Hill-Climbing

2.2.2.1 Single Ship

Tidak diimplementasikan sebagai fungsi terpisah, sehingga tidak ada hasil eksperimen.

2.2.3 Hill-Climbing with Sideways Move

2.2.3.1 Single Ship

Single ship (parameter: max_sideways=100, max_iter=10000, reset_on_improve=true)

Run	Obj. Awal	Obj. Akhir	Iterasi	Durasi (s)	Langkah sideways	Gap ke optimum
1	798	1127	124	1,25	115	5,1%
2	764	1175	113	1,13	104	1,0%
3	982	1095	115	1,16	112	7,7%
Rata-rata	848,0	1132,3 (SD 32,9)	117,3	1,18		4,6%

Catatan.

- Ketiga run berhenti karena max_sideways, bukan karena local optimum ketat.
- Nilai akhir dicapai pada iterasi 24, 13, dan 15. Tepat 100 iterasi terakhir di tiap run adalah sideways tanpa kenaikan nilai, atau 81–89% dari seluruh iterasi.
- Sideways tetap berguna di tengah lintasan. Total ada 4 plateau yang berhasil dilewati, dengan panjang 7, 8, 4, dan 12 langkah.
- Grafik: output/hc/run{1,2,3}_objective.png. State awal dan akhir: run{N}_awal.png dan run{N}_akhir.png.

2.2.4. *Random Restart Hill-Climbing*

2.2.4.1 Single Ship

Tidak diimplementasikan sebagai fungsi terpisah, sehingga tidak ada hasil eksperimen.

2.2.5 *Simulated Annealing*

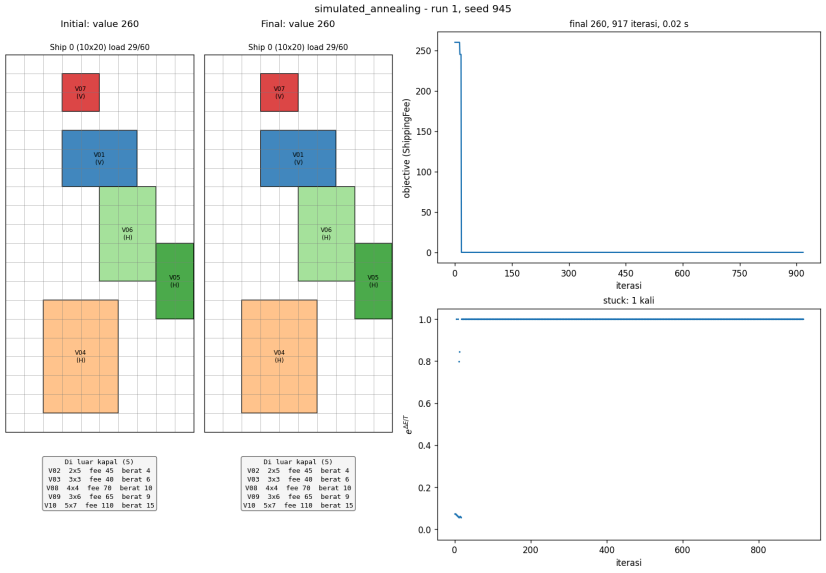
1. instance_example

Hasil JSON yang telah tersimpan di repo kelompok (commit 04720dc), untuk data/instance_example.json. Run 1–3 memakai seed 945–947, T_0=100, α=0,99, Tmin=0,01, N=50, dan masing-masing 917 iterasi. Durasi dibulatkan empat angka di belakang koma.

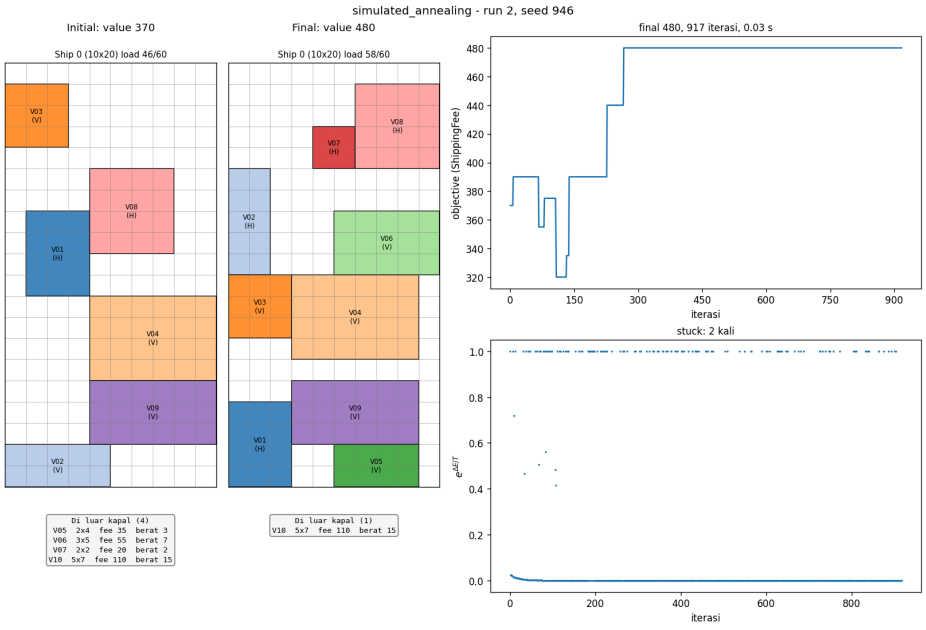
Tabel 2.2.5.1. instance_example *Simulated Annealing*

Run	***Objective* Awal**	***Objective* Akhir**	**Durasi (s)**	**Frekuensi *Stuck***
1	260	260	0,0169	1
2	370	480	0,0297	2
3	195	300	0,0160	1

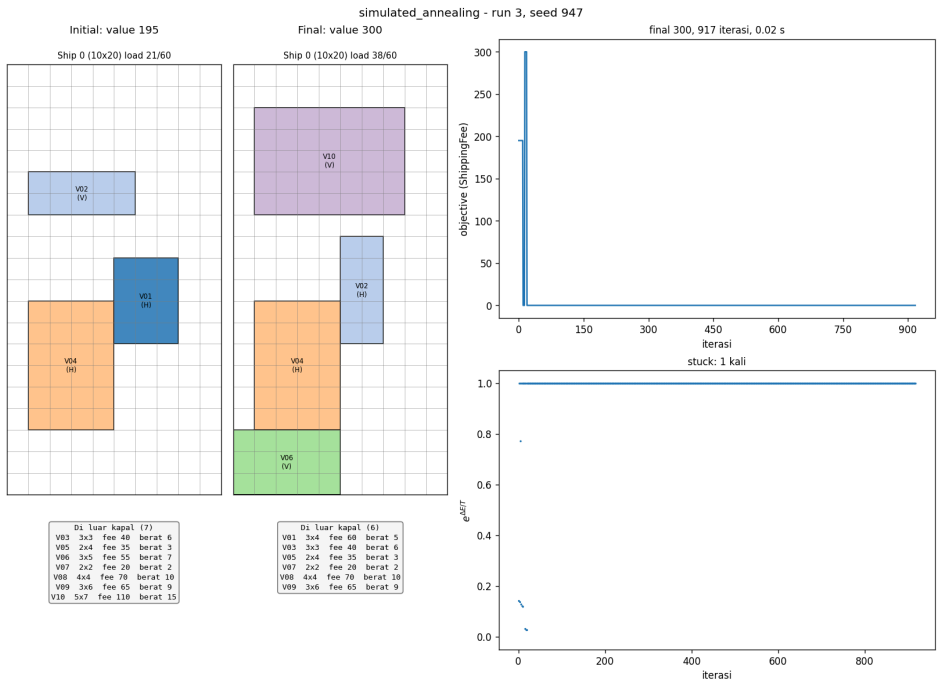
simulated_annealing - run 1, seed 945



Gambar 2.2.5.1.1. State awal, solusi valid terbaik, dan plot SA pada instance_example, seed 945: objective 260 menjadi 260; stuck 1 episode.



Gambar 2.2.5.1.2. State awal, solusi valid terbaik, dan plot SA pada instance_example, seed 946: objective 370 menjadi 480; stuck 2 episode.



Gambar 2.2.5.1.3. State awal, solusi valid terbaik, dan plot SA pada instance_example, seed 947: objective 195 menjadi 300; stuck 1 episode.

Catatan.

- Iterasi selalu 1838, bukan 5000, karena SA berhenti oleh min_temp. Angkanya cocok dengan

  $$ln(10^{-4})/ln(0,995) \approx 1837,5.$$

- Nilai akhir sama dengan nilai best di ketiga run.
- Best ditemukan terlambat, pada iterasi 1803, 854, dan 1583, saat suhu sudah sangat rendah (0,012; 1,39; 0,036).
- Awal episode stuck:
  - Run 1: iterasi 139, 569, 948, 1562.
  - Run 2: iterasi 190, 483, 954.
  - Run 3: iterasi 106, 575, 1150, 1683.

- Stagnasi terpanjang adalah 613, 984, dan 574 iterasi. Pada run 2, best terakhir tercapai di iterasi 854, lalu 984 iterasi sisanya (±53% run) tidak menambah

2.2.5.1 *Single Ship*

2.2.6 Genetic Algorithm

Genetic Algorithm (GA) adalah algoritma pencarian yang terinspirasi dari seleksi alam. Pencarian tidak dilakukan dari satu state, melainkan dari sekumpulan state (populasi) yang diperbaiki dari generasi ke generasi lewat seleksi, *crossover*, dan mutasi [1][2][3]. Berbeda dengan *Hill-Climbing* dan *Simulated Annealing* yang bergerak dari satu state ke *neighbor*-nya, GA mencari secara paralel dan dapat menggabungkan bagian-bagian baik dari dua solusi. Implementasi ada pada src/ga.py, fungsi genetic_algorithm.

2.2.6.1 *Single Ship*

a. **Representasi individu dan fitness.**

Individu adalah sebuah objek *State*, yaitu pemetaan setiap kendaraan ke (x, y, orientasi) atau *None* bila di luar kapal. *Fitness* adalah *State.value()*, yaitu total *ShippingFee* kendaraan yang diangkut, sehingga semakin besar semakin baik. Karena setiap individu dibuat dan dimodifikasi lewat operasi yang menjaga validitas, seluruh individu pada populasi selalu memenuhi batasan (tidak bertumpuk, di dalam geladak, tidak melebihi kapasitas).

b. Alur Algoritma

```
populasi <- N state acak (random_state)
ulangi sebanyak jumlah iterasi (generasi):
    urutkan populasi menurun berdasarkan fitness
    populasi baru <- 2 individu terbaik (elitisme)
    selama ukuran populasi baru < N:
        A <- seleksi tournament
        B <- seleksi tournament
        anak <- crossover(A, B)
        dengan peluang 0,3: anak <- mutasi(anak)
        tambahkan anak ke populasi baru
    populasi <- populasi baru
    catat nilai maksimum dan rata-rata populasi
kembalikan individu terbaik yang pernah ditemukan
```

1.1.6.1.3 Deskripsi Fungsi

1. pilih_parent(populasi, rng, k=3) adalah seleksi tournament. Dimana diambil k individu acak dari populasi dan yang fitness-nya terbesar menjadi parent. Tournament mudah diatur tekanan seleksinya lewat k dan tidak membutuhkan penskalaan *fitness*.
2. crossover(a, b, rng). crossover satu titik pada daftar kendaraan. Kendaraan sebelum titik potong mengikuti penempatan parent A, sisanya mengikuti parent B. Gabungan ini bisa menghasilkan kendaraan yang bertabrakan atau berat total berlebih, sehingga anak dibangun dengan State.from_plan, yang menempatkan kendaraan satu per satu dan membiarkan yang tidak muat berada di luar kapal. Urutan penempatan diacak agar bukan kendaraan yang sama yang selalu mendapat tempat lebih dulu.
3. mutasi(s, rng) untuk mengganti individu dengan satu neighbor acak (random_neighbor), yaitu tukar dua kendaraan, pindahkan kendaraan, atau ubah orientasi. Bila tidak ada neighbor valid, individu dibiarkan.

4. genetic_algorithm(vehicles, ship, jumlah_populasi, iterasi, mutation_rate=0.3, elite=2, seed=None) merupakan loop utama. Elitisme (dua individu terbaik langsung lolos) menjamin nilai maksimum populasi tidak pernah turun. Fungsi mengembalikan state awal, state akhir, riwayat nilai maksimum dan rata-rata per generasi, jumlah populasi, iterasi, dan durasi.

C. Rancangan Eksperimen

Instance terdapat 30 kendaraan, kapal 8 x 12, kapasitas 100 (total fee seluruh kendaraan 1594). Sesuai spesifikasi, ada dua percobaan, masing-masing konfigurasi dijalankan 3 kali:

1. Populasi dikontrol pada 30, iterasi divariasikan: 50, 100, 200.
2. Iterasi dikontrol pada 100, populasi divariasikan: 10, 30, 50.

Pada setiap *run* dicatat *state* awal dan akhir, nilai *objective* akhir, jumlah populasi, banyak iterasi, durasi, dan plot nilai maksimum serta rata-rata populasi terhadap iterasi (output/plots/ga/).

D. Hasil

Tabel Semua Run:

Percobaan	**Populasi**	**Iterasi**	**Run**	**Nilai awal**	**Nilai akhir**
Iterasi divariasikan	30	50	1	977	1064
	30	50	2	968	1057
	30	50	3	982	1106
	30	100	1	977	1064

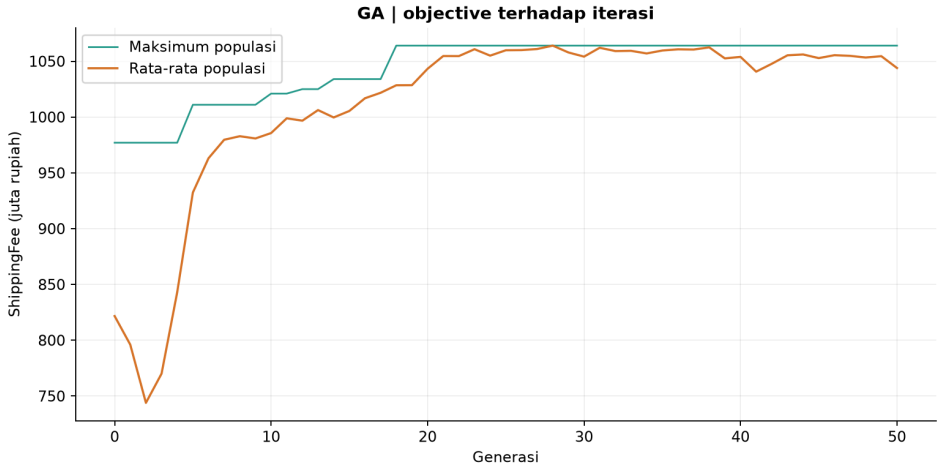
	30	100	2	968	1057
	30	100	3	982	1114
	30	200	1	977	1064
	30	200	2	968	1057
	30	200	3	982	1114
Populasi divariasikan	10	100	1	977	1056
	10	100	2	968	1109
	10	100	3	982	1045
	50	100	1	1019	1095
	50	100	2	970	1117
	50	100	3	996	1109

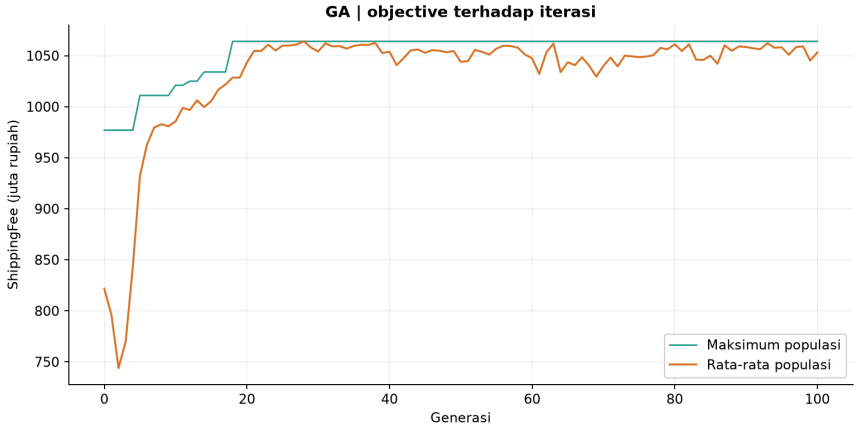
Percobaan 1: populasi dikontrol (30), iterasi divariasikan

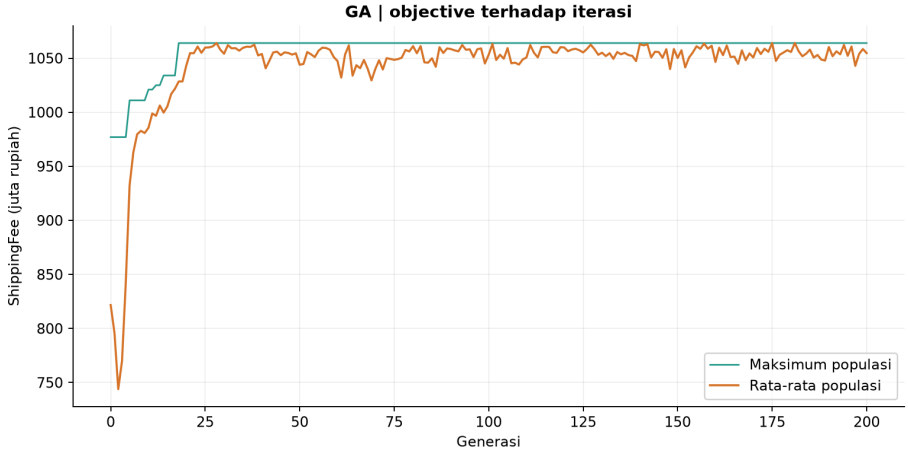
Tabel Hasil per run

Iterasi	Run	Nilai awal	Nilai akhir	Durasi (s)
50	1	977	1064	0,113
50	2	968	1057	0,119
50	3	982	1106	0,118
100	1	977	1064	0,236
100	2	968	1057	0,238
100	3	982	1114	0,216
200	1	977	1064	0,410

200	2	968	1057	0,440
200	3	982	1114	0,423


GA | objective terhadap iterasi
Maksimum populasi
Rata-rata populasi
ShippingFee (juta rupiah)
Generasi
750
800
850
900
950
1000
1050
0
10
20
30
40
50

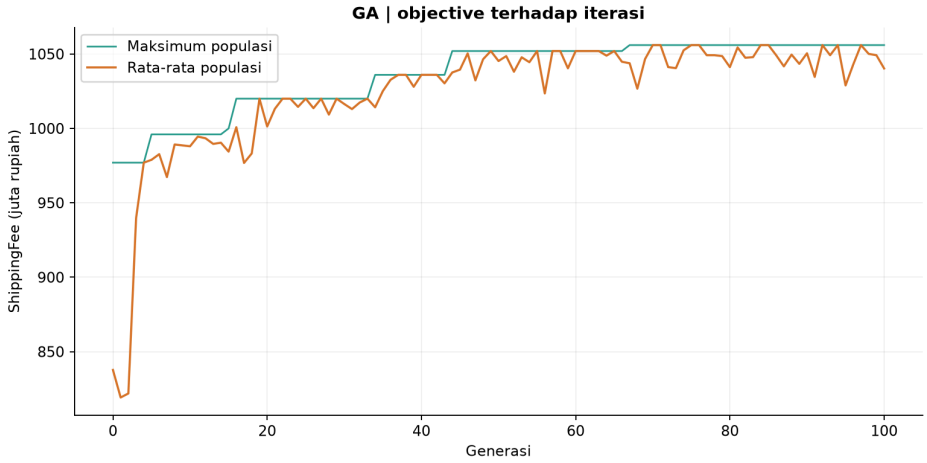

GA | objective terhadap iterasi
Maksimum populasi
Rata-rata populasi
ShippingFee (juta rupiah)
Generasi
750
800
850
900
950
1000
1050
0
20
40
60
80
100

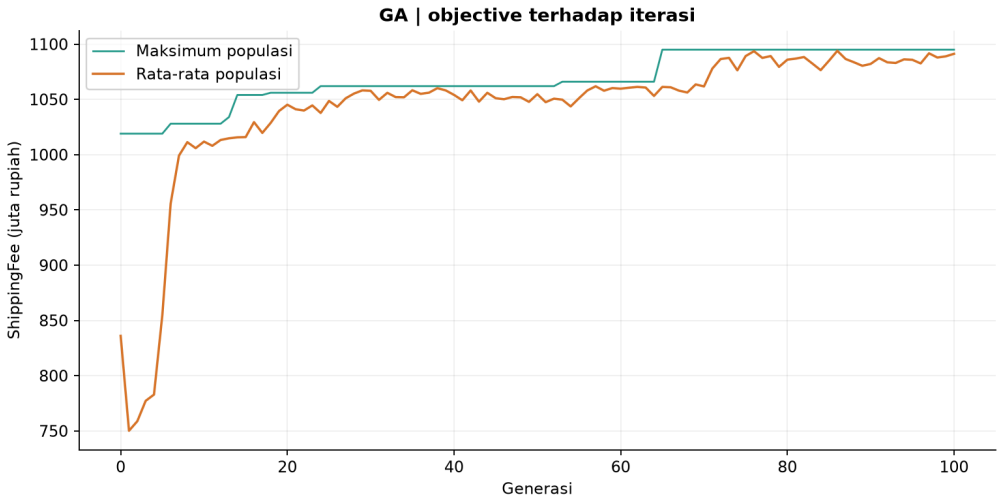

GA | objective terhadap iterasi
Maksimum populasi
Rata-rata populasi
ShippingFee (juta rupiah)
Generasi
750
800
850
900
950
1000
1050
0
25
50
75
100
125
150
175
200

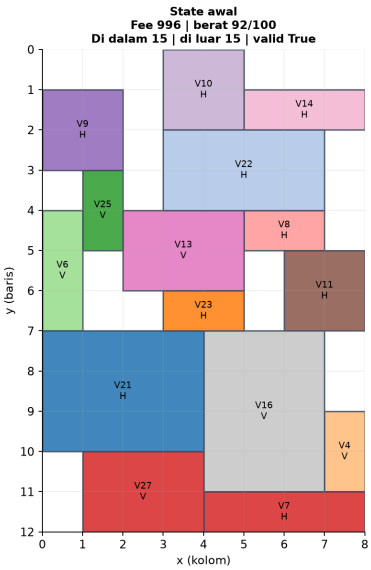
Percobaan 2: iterasi dikontrol (100), populasi divariasikan

Tabel Hasil per run

Populasi	Run	Nilai awal	Nilai akhir	Durasi (s)
10	1	977	1056	0,069
10	2	968	1109	0,063
10	3	982	1045	0,060
30	1	977	1064	0,211
30	2	968	1057	0,208

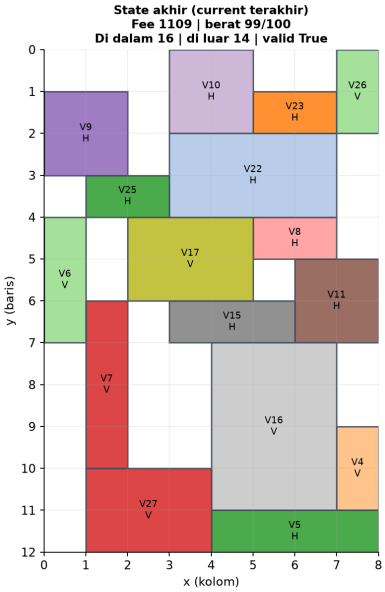






Kendaraan di luar kapal
Tidak memiliki koordinat geladak

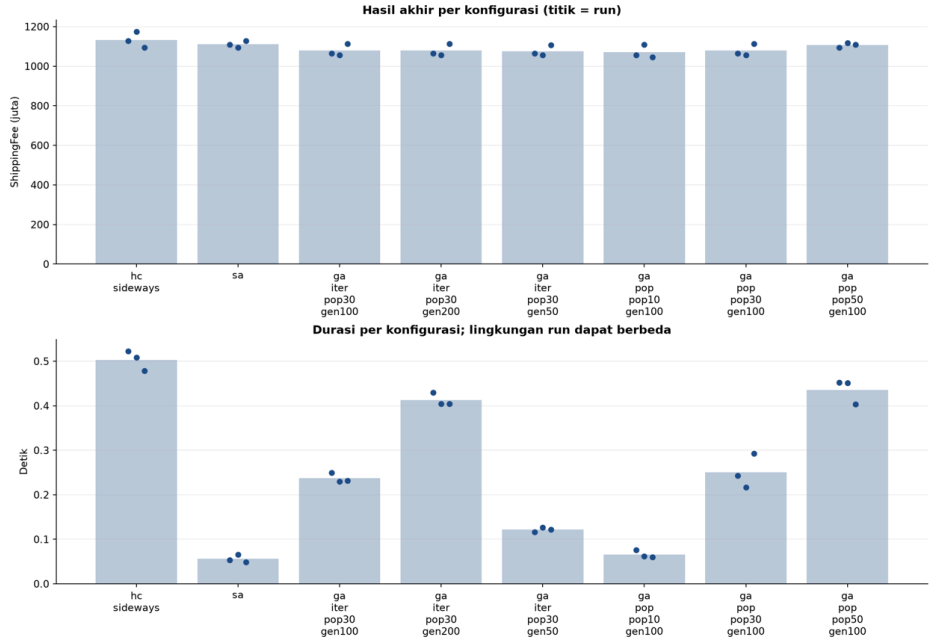
ID	w x l	Fee (juta)	Berat
V1	3 x 2	13	6
V2	1 x 2	23	10
V3	3 x 3	14	2
V5	1 x 4	93	10
V12	2 x 4	34	3
V15	1 x 3	55	5
V17	3 x 2	78	5
V18	2 x 3	44	10
V19	3 x 3	17	5
V20	2 x 3	44	3
V24	3 x 4	64	8
V26	1 x 2	29	4
V28	2 x 4	42	10
V29	3 x 4	24	10
V30	3 x 3	24	6



Kendaraan di luar kapal
Tidak memiliki koordinat geladak

ID	w x l	Fee (juta)	Berat
V1	3 x 2	13	6
V2	1 x 2	23	10
V3	3 x 3	14	2
V12	2 x 4	34	3
V13	3 x 2	47	3
V14	1 x 3	45	9
V18	2 x 3	44	10
V19	3 x 3	17	5
V20	2 x 3	44	3
V21	3 x 4	50	5
V24	3 x 4	64	8
V28	2 x 4	42	10
V29	3 x 4	24	10
V30	3 x 3	24	6

Sebagai ringkasan visual kedua percobaan, dapat dilihat plot perbandingan konfigurasi berikut:



Tabel Ringkasan GA:

Populasi	**Iterasi**	**Nilai awal (rata-rata)**	**Nilai akhir (rata-rata)**	**Rentang**	**Durasi (s)**
30	50	975,7	1075,7	1057 sampai 1106	0,117
30	100	975,7	1078,3	1057 sampai 1114	0,230
30	200	975,7	1078,3	1057 sampai 1114	0,424
10	100	975,7	1070,0	1045 sampai 1109	0,064
50	100	995,0	1107,0	1095 sampai 1117	0,447

BAB III.

ANALISIS

Algoritma	Nilai akhir tiap run	Rata-rata	Selisih terbesar antar run	Durasi rata-rata (detik)
Hill-Climbing sideways	1127, 1175, 1095	1132,3	80	0,475
Simulated Annealing	1109, 1095, 1127	1110,3	32	0,053
GA (populasi 50, 100 iterasi)	1095, 1117, 1109	1107,0	22	0,447
GA (populasi 30, 100 iterasi)	1064, 1057, 1114	1078,3	57	0,230
GA (populasi 10, 100 iterasi)	1056, 1109, 1045	1070,0	64	0,064

1. Seberapa dekat tiap algoritma dengan *global optima*, dan mengapa?

   Hasil terbaik GA adalah 1117 (populasi 50, run 2), sedangkan hasil terbaik di antara semua algoritma adalah 1175 (Hill-Climbing, run 2). Artinya GA masih menyisakan sedikitnya 58 poin dari hasil terbaik yang pernah ditemukan, dan optimum sebenarnya mungkin lebih tinggi lagi. GA memperbaiki state awalnya secara konsisten: rata-rata nilai awal 975,7 (populasi 30) naik menjadi 1078,3, dan 995,0 (populasi 50) naik menjadi 1107,0.

2. Bagaimana perbandingan hasil antar algoritma?

Seperti terlihat pada Tabel diatas, pada instance ini Hill-Climbing memberi rata-rata tertinggi (1132,3), disusul Simulated Annealing (1110,3) dan GA terbaik (1107,0). Selisih Simulated Annealing dan GA hanya 3 poin, jauh lebih kecil daripada selisih antar run (22 sampai 32 poin), sehingga keduanya dianggap setara dan tidak dapat dikatakan salah satunya lebih baik. Selisih antara Hill-Climbing dan dua algoritma lainnya (sekitar 22 sampai 25 poin) juga hanya sedikit lebih besar daripada selisih antar run. Hill-Climbing mengevaluasi seluruh neighbor pada setiap langkah sehingga pencariannya sangat terarah. GA hanya mencoba satu neighbor acak pada mutasi, dan crossover satu titik dapat merusak susunan penempatan yang sudah baik karena kendaraan yang bergantung pada posisi relatif satu sama lain terpisah. Pada GA terdapat juga mekanisme perbaikan (from_plan) yang membuang kendaraan yang bentrok, sehingga anak bisa kehilangan fee dari kendaraan yang terbuang.

3. Bagaimana perbandingan durasi pencarian antar algoritma?

   Simulated Annealing paling cepat (sekitar 0,05 detik) walau menjalankan 1838 iterasi, karena setiap iterasi hanya mengevaluasi satu neighbor. Hill-Climbing sekitar 0,48 detik dengan 113 sampai 124 iterasi karena setiap iterasi mengevaluasi seluruh neighbor. GA berkisar antara 0,06 detik (populasi 10) dan 0,45 detik (populasi 50) untuk 100 iterasi. Pada GA, durasi hampir sebanding dengan ukuran populasi dikali banyak iterasi: populasi 50 sekitar dua kali lebih lama daripada populasi 30 pada iterasi yang sama, dan 200 iterasi sekitar dua kali lebih lama daripada 100 iterasi. Waktu diukur di satu komputer tanpa mengontrol beban proses lain, sehingga angka sekecil ini sebaiknya dibaca sebagai urutan besaran, bukan selisih yang pasti.

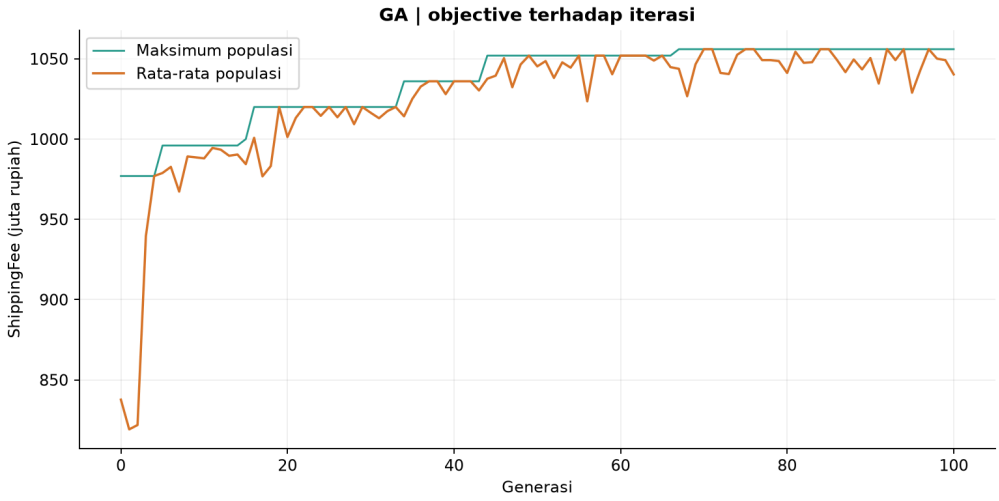
4. Seberapa konsisten hasil akhir antareksperimen?

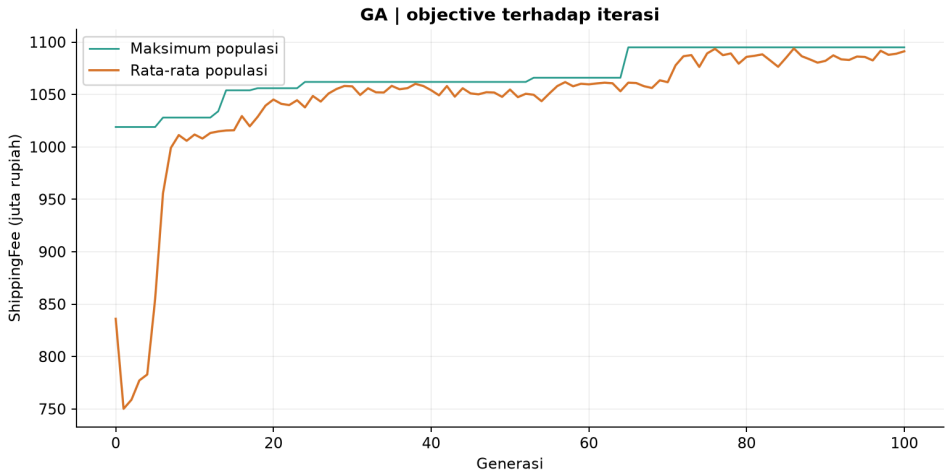
   Selisih terbesar antar run adalah 80 poin pada Hill-Climbing, 32 poin pada Simulated Annealing, dan pada GA berkisar dari 22 poin (populasi 50) sampai 64 poin (populasi 10). Populasi yang lebih besar menghasilkan

hasil yang lebih stabil, kemungkinan karena keragaman populasi yang lebih besar mengurangi pengaruh keberuntungan pada populasi awal. Hill-Climbing paling bervariasi karena hanya menerima langkah yang tidak memburuk sehingga hasil akhirnya bergantung pada state awal.

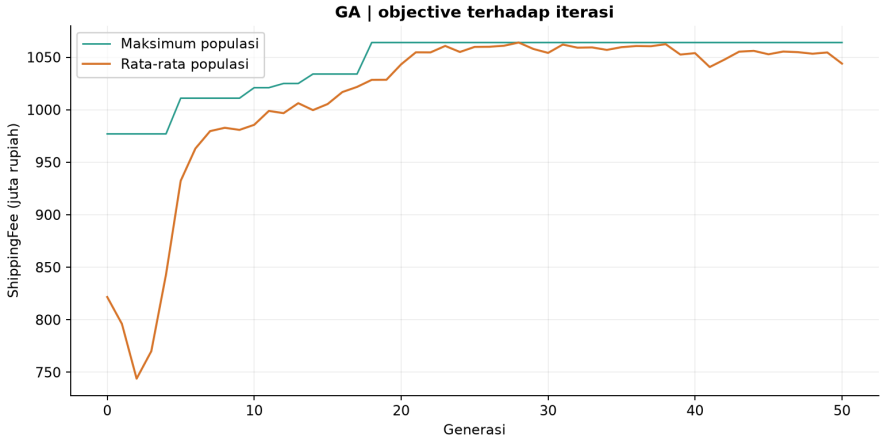
5. Bagaimana pengaruh banyak iterasi dan jumlah populasi pada GA?

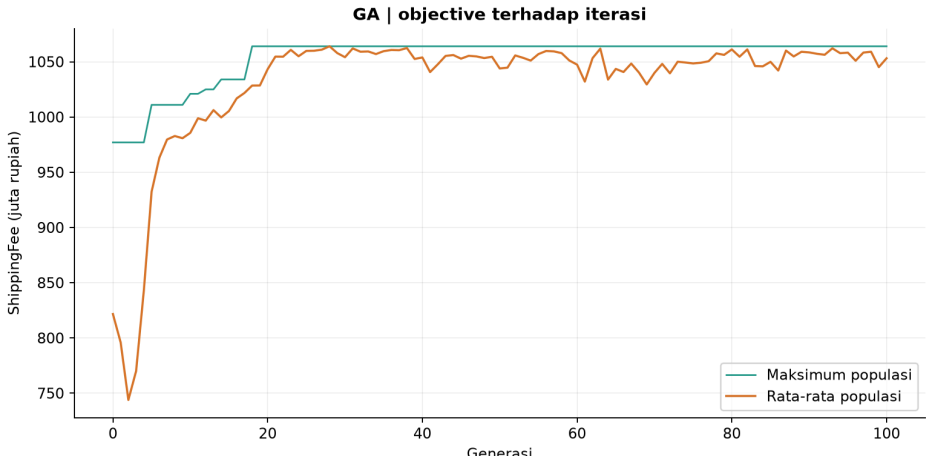
Jumlah populasi. Rata-rata nilai akhir naik seiring populasi: 1070,0 (populasi 10), 1078,3 (populasi 30), dan 1107,0 (populasi 50), pada iterasi yang sama (100). Populasi yang lebih besar memberi cakupan ruang pencarian yang lebih luas dan peluang lebih besar menemukan individu yang baik, tetapi biayanya waktu yang naik kira-kira sebanding. Pada populasi 50 selisih antar run juga paling kecil.

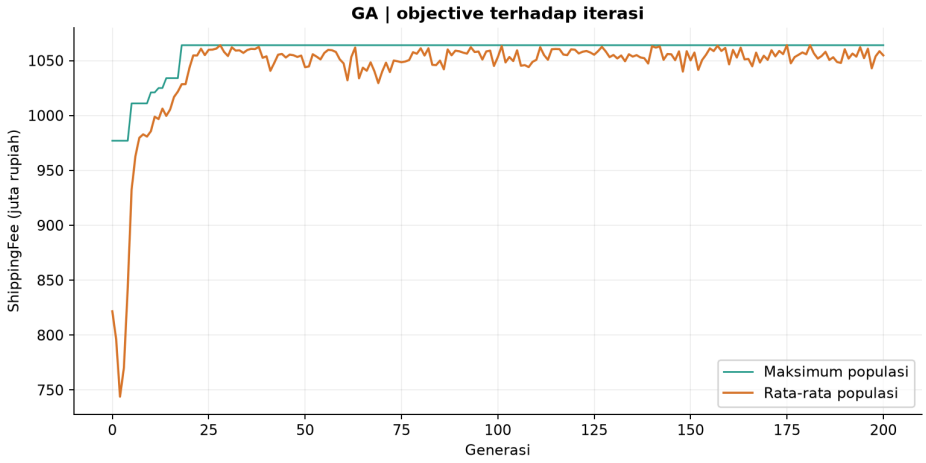


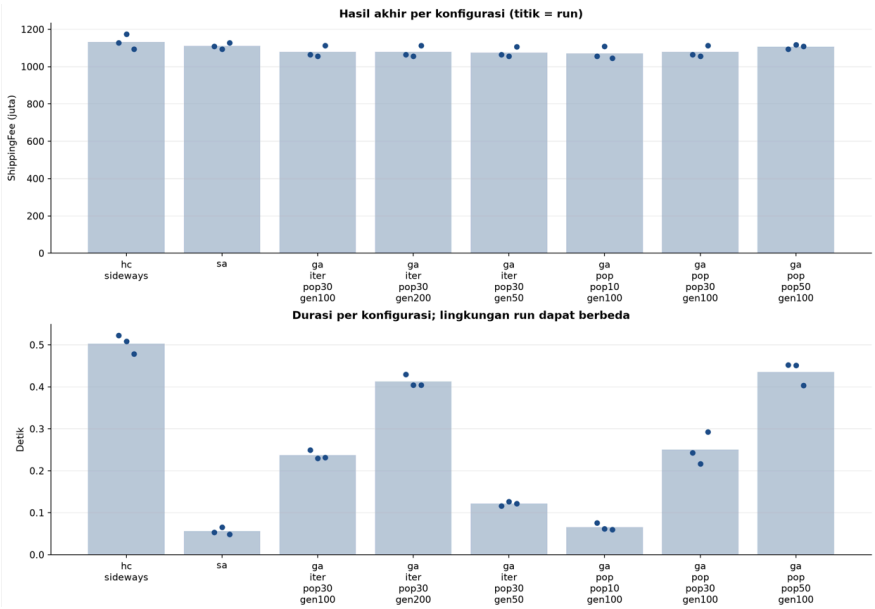


Banyak iterasi. Dalam rentang 50, 100, dan 200 iterasi pada populasi 30, rata-rata nilai akhir hampir tidak berubah (1075,7, 1078,3, dan 1078,3). Pada run 1 dan run 2, nilai akhir identik di ketiga variasi (1064 dan 1057), dan hanya run 3 yang bergeser dari 1106 menjadi 1114 antara 50 dan 100 iterasi. Artinya pencarian sudah berhenti membaik sebelum iterasi ke-50 pada sebagian besar run, yang menandakan konvergensi dini. Penyebabnya adalah keragaman populasi cepat menyempit karena seleksi *tournament* dan elitisme dua individu, sehingga turunan semakin mirip dan *crossover* antar *parent* yang mirip tidak menghasilkan susunan baru. Ini dugaan dan belum diuji.









6. Pertanyaan tambahan

a. Apakah mekanisme SA benar-benar berperan? Penerimaan langkah turun hanya 33–48 kali per *run* (1,8–2,6% iterasi). Namun langkah *sideways* diterima di ± 70% iterasi, dan *best* ditemukan pada suhu sangat rendah di run 1 dan 3 (T ≈ 0,012 dan 0,036), saat peluang menerima penurunan berukuran satu fee kendaraan (≥ 13 juta) praktis nol. Artinya, perbaikan akhir SA berasal dari penjelajahan *plateau* dan langkah naik, bukan dari penerimaan penurunan di suhu rendah. Fase suhu tinggi (± 500 iterasi pertama) berfungsi

sebagai eksplorasi awal, lalu SA bekerja mirip *stochastic hill-climbing*dengan *sideways*. Selisih fee antar kendaraan besar (13–93) dibanding T_0 = 100, sehingga suhu efektif hanya berpengaruh di fase awal. Penjadwalan yang lebih panjang di rentang T = 1–20 mungkin lebih berguna.

b. Apakah max_sideways = 100 tepat? Pada ketiga *run* HC, tidak ada perbaikan sama sekali dalam 100 langkah *sideways* terakhir. Nilai yang lebih kecil (mis. 10–30) kemungkinan memberi hasil sama dengan waktu jauh lebih pendek, tetapi ini hipotesis yang perlu dibuktikan.

c. Mengapa frekuensi *stuck* SA tidak sama dengan jumlah *local optimum*? Karena kode hanya memantau panjang stagnasi *best* (4, 3, 4 episode), tanpa memeriksa *neighborhood*. Angkanya hanya sebagai indikator.

BAB III.

KESIMPULAN DAN SARAN

3.1. Kesimpulan

1. Ketiga algoritma menghasilkan penataan valid (dalam batas geladak, tanpa overlap, berat ≤ 100). Seluruh 24 hasil lolos validasi.
2. Hasil terbaik adalah 1.175 juta (HC run 2). Batas atas yang dihitung dengan DP adalah 1.187, sehingga optimum global ada di rentang 1.175–1.187, dan hasil terbaik paling jauh 1,0% dari optimum.
3. HC rata-rata tertinggi (1.132,33) tetapi paling bervariasi (SD 40,27) dan paling lambat (1,18 s). SA rata-rata 1.110,33 dengan variasi lebih kecil dan durasi tercepat (0,039 s). GA terbaik (P50/G100) rata-rata 1.107 dengan variasi terkecil (SD 11,14). Karena hanya 3 *run* dan rentang hasilnya tumpang tindih, perbedaan HC dan SA belum signifikan.
4. Pada GA, populasi lebih besar memperbaiki rata-rata dan konsistensi, sedangkan generasi di atas 100 tidak menambah hasil pada populasi 30.
5. Berat adalah batasan yang paling mengikat. Sembilan kendaraan bernilai tinggi dipilih oleh semua *run*, dan perbedaan hasil ditentukan oleh susunan 45 unit berat sisanya.
6. Temuan berlaku untuk satu instance dan tiga *run* per konfigurasi. Varian HC lain dan multi-kapal tidak diimplementasikan.

Dari pengujian Simulated Annealing, penyimpanan best feasible memastikan solusi akhir memenuhi constraint. Konfigurasi suhu awal 100 memberikan hasil yang bergantung pada seed dan karakteristik instance. Salah satu run instance_multiship mencapai total fee seluruh kendaraan, yaitu 590, sedangkan pada beberapa instance lain pencarian lama berada pada plateau nilai 0. Pengujian menegaskan pentingnya pemilihan suhu serta pemisahan jumlah penolakan neighbor dari stagnasi nilai terbaik.

Pada pengujian Hill-Climbing, steepest ascent dan stochastic cepat berhenti ketika tidak ada neighbor yang memperbaiki objective. Sideways move membantu melewati plateau, khususnya pada instance_multiship_hard dan instance_tight_space, sementara random restart mengurangi ketergantungan pada satu state awal. Steepest ascent, sideways move,

dan random restart mencapai optimum 590 pada seluruh run instance_multiship. Seluruh 72 solusi akhir HC memenuhi constraint. Hasil pada instance lain menunjukkan kompromi antara kualitas solusi, variasi hasil, dan usaha pencarian; tidak ada varian yang selalu terbaik.

3.2 Saran

Pengujian SA berikutnya dapat memvariasikan suhu awal dan cooling rate pada seluruh instance dengan anggaran iterasi yang setara. Fungsi evaluasi yang membedakan tingkat pelanggaran constraint dapat diteliti untuk mengurangi plateau nilai 0, dengan tetap memastikan solusi akhir feasible dan menjelaskan heuristik yang digunakan. Perbandingan durasi algoritma sebaiknya dijalankan pada perangkat yang sama, sedangkan penilaian kedekatan terhadap global optimum dapat dibantu batas atas atau solusi eksak pada instance kecil.

Untuk Hill-Climbing, max_sideways dan max_restart dapat divariasikan dengan anggaran evaluasi neighbor atau waktu yang setara agar perbandingan lebih adil. Pengujian dengan lebih banyak seed serta pencatatan jumlah evaluasi objective dapat membedakan pengaruh strategi pencarian dari biaya tiap iterasi. Instance kecil dapat dibandingkan dengan solusi eksak, sedangkan batas atas objective dapat digunakan untuk mengukur selisih terhadap optimum ketika solusi eksak belum tersedia.

4. Pembagian Tugas

Nama	NIM	Kontribusi
Dastya	11523033	1. Genetic Algorithm, Constraint Validation, 3 move, initial state, class state

Stella Cometta Febriana	1822435	1. Implementasi stimulate annealing 2. Penyusunan laporan deskripsi persoalan , pembahasan simplementasi (2.1) 3. Penyusunan semua plot, visualisasi geladak
Nathan Pasha Athalla	1824053	

Form Penggunaan *Generative AI*

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM : 1. Daysta / 11523033
2. 18224035/ Stella Cometta Febriana
3. Nathan Pasha Athalla/ 18224053

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial

Menyatakan bahwa ~~saya~~/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya/Tidak*	5
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya/Tidak*	7

Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/Tidak*	7
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya/Tidak*	0
Lainnya, Jelaskan: Penggunaan AI dalam kode, terutama dalam debugging dan cek redundansi pada kode	Ya/Tidak*	7

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026

(tanda tangan) 18224035

Nathan Pasha Athalla

18224053

Daysta 11523033

Stella Cometta Febriana